



Tarefa 4

Ferramentas para desenvolvimento

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis

PEDs: Caroline Nakazato, Giovane Mariano, Gustavo Morais e
Marcos Freire

PADs: Caio, Lia, Paulo e Vinicius

1. Descrição Geral

Até este ponto da disciplina, você implementou gradualmente um sistema de combate inspirado em *Slay the Spire*, adicionando novas funcionalidades a cada laboratório. O projeto agora possui diversas classes, uma hierarquia de objetos mais rica e múltiplos sistemas interagindo entre si, como baralho de cartas, efeitos e fluxo de combate.

Em projetos reais de software, conforme o sistema cresce, torna-se importante utilizar ferramentas que auxiliem na **organização, compilação e documentação do código**. Ferramentas de *build* permitem automatizar tarefas como compilação e execução do programa, enquanto sistemas de documentação ajudam a descrever claramente o funcionamento das classes e métodos do projeto.

Neste laboratório, você irá dar mais um passo na evolução do projeto ao introduzir duas tecnologias amplamente utilizadas no desenvolvimento em Java: **Gradle** e **Javadoc**. Além disso, você continuará expandindo o jogo desenvolvido ao longo da disciplina por meio da implementação de novas cartas.

1.1. Uso de Gradle

Até agora, o projeto foi compilado diretamente utilizando comandos como `javac`. Embora esse método funcione para projetos pequenos, ele rapidamente se torna difícil de manter conforme o número de arquivos cresce.

Neste laboratório, você irá refatorar seu projeto para utilizar **Gradle**, uma ferramenta de automação de *build* amplamente utilizada no ecossistema Java. Ferramentas de *build* ajudam a organizar e automatizar diversas tarefas comuns no desenvolvimento de software, reduzindo a necessidade de executar comandos manualmente.

Além de compilar o projeto, ferramentas como o Gradle permitem:

- **Automatizar tarefas repetitivas**, como compilação, execução e geração de artefatos;
- **Gerenciar dependências automaticamente**, baixando bibliotecas necessárias sem que seja preciso adicioná-las manualmente ao projeto;
- **Executar testes automatizados** de forma padronizada;
- **Padronizar o processo de build**, garantindo que diferentes desenvolvedores consigam compilar e executar o projeto da mesma forma.

Esse tipo de ferramenta é extremamente comum em projetos reais de software e ajuda a evitar problemas como a situação conhecida como “*funciona na minha máquina*”, que pode ocorrer quando diferentes ambientes possuem configurações ou dependências distintas.

Ao final deste laboratório, seu projeto poderá ser compilado e executado utilizando comandos do Gradle, tornando o processo de desenvolvimento mais organizado e próximo do que é utilizado no mercado de trabalho.

1.2. Documentação com Javadoc

Outro aspecto fundamental de projetos de software é a **documentação do código**. Em projetos maiores, outros desenvolvedores precisam entender rapidamente o propósito de cada classe e método.

O **Javadoc** é a ferramenta padrão do ecossistema Java para geração automática de documentação a partir de comentários presentes no código-fonte. Neste laboratório, você irá documentar as principais classes e métodos do seu projeto e gerar a documentação correspondente.

1.3. Expansão do jogo

Por fim, você continuará expandindo o jogo desenvolvido ao longo da disciplina. Aproveitando a estrutura orientada a objetos construída nos laboratórios anteriores, você deverá implementar novas cartas utilizando os sistemas já existentes, como dano, escudo e efeitos.

Esse processo ajuda a demonstrar uma das principais vantagens da programação orientada a objetos: a capacidade de **estender sistemas existentes com novas funcionalidades sem alterar significativamente o código já implementado**.

2. Objetivos

Ao final deste laboratório, o aluno deverá ser capaz de:

- Refatorar um projeto Java existente para utilizar uma ferramenta de **build automation** (Gradle);
- Utilizar **Javadoc** para documentar classes e métodos de um projeto;
- Gerar documentação automaticamente a partir dos comentários presentes no código;
- Expandir um sistema orientado a objetos existente adicionando novas classes e comportamentos;
- Reutilizar e estender funcionalidades já implementadas por meio de **herança** e **polimorfismo**.

3. Atividades

Esta seção descreve as atividades do laboratório. O objetivo é evoluir o projeto desenvolvido até o momento introduzindo ferramentas comuns no desenvolvimento de software em Java e expandindo o jogo com novas funcionalidades.

Assim como nos laboratórios anteriores, as instruções apresentadas abaixo definem os requisitos mínimos esperados. No entanto, você é livre para adicionar novas classes, atributos ou métodos sempre que isso ajudar a melhorar a organização ou a clareza do código.

As seguintes diretrizes gerais devem ser observadas:

- **Princípios de POO:** Continue aplicando os conceitos de orientação a objetos vistos ao longo da disciplina. Sempre que possível, reutilize estruturas já existentes no projeto (como hierarquias de classes e efeitos) em vez de criar soluções específicas para cada novo caso;
- **Organização do Projeto:** O uso do Gradle introduz uma estrutura de projeto padronizada. Procure manter seu código organizado dentro dessa estrutura e evitar duplicação desnecessária de código;
- **Liberdade Criativa:** A expansão do jogo é aberta. Você pode criar novas cartas, efeitos, inimigos ou mecânicas que se integrem ao sistema já implementado;
- **Autonomia Técnica:** Alguns detalhes de implementação são intencionalmente deixados em aberto. Espera-se que você:
 - Escolha tipos de variáveis adequados para cada atributo;
 - Defina parâmetros apropriados para cada método;
 - Tome decisões de design coerentes com os princípios de orientação a objetos.

3.1. Refatoração do Projeto para Gradle

A primeira etapa deste laboratório é refatorar seu projeto para utilizar **Gradle** como ferramenta de build. Para isso, você pode utilizar o suporte a Gradle disponível no VSCode ou inicializar o projeto manualmente utilizando o comando:

```
gradle init
```

Durante a inicialização, selecione a opção de projeto **Java Application**.

A estrutura típica de um projeto Gradle é semelhante a:

```

.
|-- build.gradle
|-- settings.gradle
|-- gradlew
|-- gradlew.bat
|-- src/
|   |-- main/
|   |   |-- java/
|   |   |-- ...
|   |-- test/
|       |-- java/

```

Após a inicialização do projeto:

- Mova os arquivos `.java` do seu projeto para o diretório `src/main/java`;
- Ajuste o projeto para que ele compile corretamente utilizando o Gradle.

Após a refatoração, o projeto deve poder ser compilado utilizando:

```
./gradlew build
```

e executado utilizando:

```
./gradlew run
```

Certifique-se de que o projeto compila e executa corretamente utilizando esses comandos.

3.2. Documentação com Javadoc

A segunda etapa do laboratório é documentar o projeto utilizando **Javadoc**.

O objetivo desta etapa é tornar o funcionamento do sistema mais claro para outros desenvolvedores que precisem ler ou utilizar seu código.

Utilize comentários no formato Javadoc para documentar:

- As principais classes do projeto;
- Métodos importantes da lógica do jogo;
- Atributos que não sejam imediatamente óbvios.

Os comentários devem descrever brevemente:

- O propósito da classe ou método;
- O significado dos parâmetros, quando aplicável;
- O comportamento esperado do método.

Após adicionar a documentação, gere a documentação HTML utilizando o comando apropriado do Gradle.

3.3. Expansão do Jogo

A etapa final do laboratório consiste em expandir o jogo implementando novas cartas que utilizem os sistemas já existentes, como dano, escudo e efeitos.

Devem ser implementadas pelo menos:

- **5 novas cartas;**
- Pelo menos **1 carta que aplique ou interaja com efeitos.**

As cartas podem reutilizar a infraestrutura já implementada nas classes existentes. Sempre que possível, utilize herança e polimorfismo para evitar duplicação de código.

Você é livre para definir:

- os nomes das cartas;
- seus custos em energia;
- seus efeitos e comportamentos.

O funcionamento dessas cartas deve ficar claro pela saída do programa no terminal e também deve ser descrito brevemente no **README** do projeto.

3.4. Desafio Extra (+0,5 ponto extra)



Figura 1: Uma saída formatada com texto colorido, arte ASCII e bom espaçamento

Seu sistema de combate deve estar funcionando adequadamente, mas a quantidade de sistemas adicionados até o momento pode ter deixado-o denso e difícil de entender rapidamente. Como desafio extra neste laboratório, você deve modificar a sua saída para deixá-la bonita e fácil de entender. Um exemplo de uma saída interessante e organizada é mostrada na Figura 1. Na prática, pontos serão concedidos com base na presença de aspectos como:

- [Arte ASCII](#) para ilustrar personagens;
- Uso de texto colorido para chamar atenção para partes diferentes da saída;
- Limpeza do terminal entre turnos;
- Pausa entre prints para dar tempo para o jogador ler;
- Logs explicativos dos acontecimentos a cada turno;
- Além de qualquer outra ideia que deixe a saída mais compreensível e visualmente interessante.

A organização do sistema de print também será avaliada. Para evitar strings longas e complicadas no meio do código, recomenda-se guardar elementos como arte ASCII e descrições longas em arquivos `.txt` separados que podem ser acessados pelo código durante o jogo.

Este desafio é inteiramente opcional, mas concluí-lo corretamente vale até +0,5 pontos extra na nota deste laboratório.

4. Avaliação

A nota do laboratório será composta pelos seguintes critérios:

1. Configuração do Projeto com Gradle (3,0 pontos)

- O projeto foi corretamente refatorado para utilizar Gradle;
- O código-fonte está organizado na estrutura padrão do Gradle (ex: `src/main/java`);
- O projeto compila corretamente utilizando o comando `./gradlew build`;
- O programa pode ser executado utilizando `./gradlew run`.

2. Expansão do Jogo e Uso de POO (3,0 pontos)

- Implementação de pelo menos **5 novas cartas**;
 - Pelo menos uma carta deve aplicar ou interagir com efeitos;
 - As novas cartas utilizam corretamente a infraestrutura já existente no projeto;
 - Uso adequado de herança, polimorfismo e reutilização de código.
3. **Documentação com Javadoc (2,0 pontos)**
- As principais classes do projeto estão documentadas utilizando comentários Javadoc;
 - Métodos relevantes possuem descrição adequada de seu comportamento;
 - A documentação pode ser gerada corretamente utilizando as ferramentas do Gradle.
4. **Organização do Projeto e Qualidade do Código (2,0 pontos)**
- Cada classe está em seu próprio arquivo `.java`;
 - Código legível, bem formatado e com nomes adequados;
 - Uso consistente de encapsulamento;
 - Comentários quando necessários para trechos não triviais.
5. **Desafio extra (até +0,5 ponto extra)**
- Conclusão correta do desafio proposto na Seção [3.4](#).

6. **Penalidade por Não Compilação**

Caso o projeto:

- Não compile utilizando `./gradlew build`; ou
- Não execute corretamente utilizando `./gradlew run`;

será aplicada uma penalidade de **-5,0 pontos** sobre a nota obtida.

A nota final mínima é **0,0** (não haverá notas negativas).

5. Entrega

A submissão é realizada exclusivamente via repositório no **GitHub**. Para isso:

- Gere uma **release (tag)** com a identificação do laboratório, seguindo o formato: `arefaX-RA1-RA2`;
- Gere uma release no GitHub a partir dessa tag;
- Envie o link da release no Google Classroom.

Caso optem por manter o repositório **privado**, adicionem os PEDs da disciplina como colaboradores:

- Caroline Nakazato – `@CarolineNakazato`
- Giovanne Mariano – `@giolucasd`
- Gustavo Moraes – `@moraguma`
- Marcos Freire – `@marcfreir`

Lembrem-se que:

- O prazo de submissão é até o dia **06/04/2026 às 23h59**.
- Entregas atrasadas serão penalizadas em **25 % da nota obtida**. Não serão aceitas entregas atrasadas após 13/04/2026.
- Os horários de laboratório e atendimento podem (e devem) ser utilizados para tirar dúvidas relacionadas tanto à submissão quanto ao desenvolvimento do laboratório.

5.1. Compilação e Execução

Durante a avaliação, os PEDs utilizarão os seguintes comandos, a partir da raiz do repositório, para compilar e executar o projeto:

```
./gradlew build  
./gradlew run
```

A classe principal do projeto deve conter o método:

```
public static void main(String[] args)
```

Essa classe será utilizada como ponto de entrada para a execução do programa. Certifique-se também de que ela está corretamente configurada no arquivo de build do Gradle para permitir a execução do comando **run**.

Dessa forma, a compilação e a execução ocorrerão corretamente no ambiente de avaliação.